Group: Sara Martinez Rendon (1000424179) | Heider Zapata (1000009586) | Yeison Londoño (1000009479)
Master's Degree in Data Sciences | EAFIT University
Course: Applied Mathematics
Date: 07/03/2026

Trabajo Final - Matemáticas Aplicadas en Ciencia de Datos

Datos

> Utilizaremos el Olivetti Faces Dataset (fácilmente disponible vía sklearn.datasets.fetch_olivetti_faces en Python). Este conjunto contiene 400 imágenes en escala de grises de 64×64 píxeles, correspondientes a 40 sujetos distintos (10 imágenes por persona). Cada imagen deberá ser "aplanada" a un vector $x \in R^{4096}$, formando nuestra matriz de diseño principal X de dimensiones 400×4096.
>
> Junto con las imágenes, la función de descarga proporcionará un arreglo unidimensional de etiquetas (target) que contiene valores enteros del 0 al 39. Cada valor en este arreglo indica exactamente a cuál de los 40 sujetos pertenece la fila correspondiente en la matriz X, una herramienta indispensable para filtrar y estructurar el problema de clasificación más adelante.
>
> Para evaluar las técnicas de detección de anomalías, inyectaremos una imagen intrusa determinista. Deberán crear mediante código un vector en R^{4096} donde la primera mitad de los elementos tenga un valor de 1.0 (blanco) y la segunda mitad tenga un valor de 0.0 (negro). Esta estructura artificial nos garantizará un comportamiento atípico en el análisis estadístico.

Primero, necesitamos descargar el dataset Olivetti Faces. Este contiene 400 imágenes de 64x64 píxeles (que aplanaremos a vectores de 4096 elementos

In [63]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces #carga olivetti faces dataset
```

In [64]:
```
# Descargamos el dataset completo
print("Descargando dataset...")
dataset = fetch_olivetti_faces(shuffle=True, random_state=42) #shuffle=True => mezcla el orden de

# Extraemos la matriz X y las etiquetas y
# Las imágenes ya vienen aplanadas en la propiedad '.data'
X = dataset.data        # Matriz de 400 x 4096 (nuestras imágenes aplanadas)
```

```
y = dataset.target      # # Arreglo de 400 etiquetas (0 al 39) que indican a qué sujeto pertenece
images = dataset.images # la propiedad '.images' permite ver las imágenes en su formato original

print("")
print(f"Dimensión de los tensores que albergan las imagenes originales: {images.shape}")
print(f"Dimensión de la matriz X (con las imagenes ya aplanadas): {X.shape}")
print(f"Dimensión de las etiquetas y: {y.shape}")
```

```
Descargando dataset...

Dimensión de los tensores que albergan las imagenes originales: (400, 64, 64)
Dimensión de la matriz X (con las imagenes ya aplanadas): (400, 4096)
Dimensión de las etiquetas y: (400,)
```

Tarea 1: El Espacio Original y Métricas de Similitud

(a) Seleccionen una imagen “ancla” de un sujeto. Utilizando operaciones vectoriales puras, calculen la distancia Euclidiana (L2) y la similitud del Coseno (derivada del producto interno y la norma) entre esta ancla y el resto de las imágenes del dataset.

En este punto, trataremos cada imagen como un punto en un hiperespacio gigantesco de 4096 dimensiones ($X \in R^{4096}$). Queremos ver qué tan bien funcionan las matemáticas básicas para saber si dos caras se parecen. Vamos a tomar la primera imagen como nuestra "ancla" y calcularemos las métricas operando directamente sobre las matrices y vectores con `numpy`.

- Distancia Euclidiana (L_2): Mide la magnitud de la diferencia espacial directa entre dos vectores x e y.

$$d_{L2}(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- Similitud del Coseno: Mide el ángulo entre dos vectores, ignorando su magnitud. Se calcula con el producto punto dividido por la magnitud de ambos vectores.

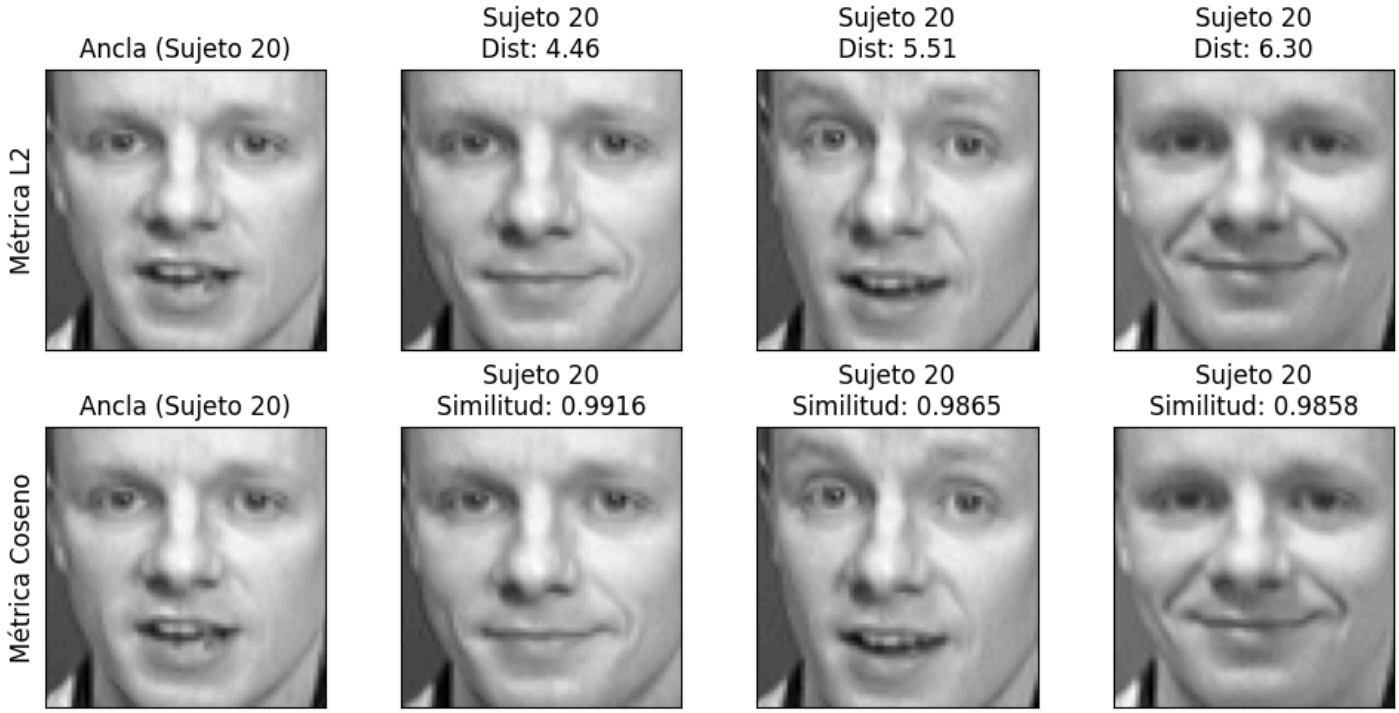
$$\text{Similitud}(x, y) = \cos(\theta) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}$$

In [65]:
```
# --- MÉTRICAS DE SIMILITUD ---

# Seleccionamos la imagen índice 0 como nuestra ancla
idx_ancla = 0
ancla = X[idx_ancla]

# 1. Distancia Euclidiana (L2) - Pura operación vectorial
# Restamos el ancla a todas las filas de X, elevamos al cuadrado, sumamos por fila y sacamos raíz
distancias_l2 = np.sqrt(np.sum((X - ancla)**2, axis=1))

# Similitud del Coseno
# Fórmula: (A dot B) / (||A|| * ||B||)
# Calculamos el producto punto del ancla con todas las demás imágenes
producto_punto = np.dot(X, ancla)
```

```python
# Calculamos las normas (magnitudes)
norma_ancla = np.linalg.norm(ancla)
normas_X = np.linalg.norm(X, axis=1)

# Similitud del coseno (valores más cercanos a 1 significan mayor similitud)
similitud_coseno = producto_punto / (normas_X * norma_ancla)
```

(b) Mediante código, encuentren las 3 imágenes más cercanas al ancla según la métrica L2 y las 3 más cercanas según el Coseno. Visualícenlas en una figura con dos filas.

In [66]:
```python
# --- ENCONTRAR LAS MÁS CERCANAS ---

# argsort devuelve los índices ordenados.
# Para L2 queremos los valores más PEQUEÑOS (menor separación física entre sí)
# [1:4] porque el índice 0 será el ancla misma (distancia 0)
top3_l2_idx = np.argsort(distancias_l2)[1:4]

# Para Coseno queremos los valores más GRANDES (cercanos a 1 -> cos(0°) = 1 -> alta similitud).
# argsort ordena de menor a mayor, así que invertimos el arreglo con [::-1]
top3_coseno_idx = np.argsort(similitud_coseno)[::-1][1:4]

# --- VISUALIZACIÓN ---
fig, axes = plt.subplots(2, 4, figsize=(10, 5))

# Título de las filas
axes[0, 0].set_ylabel("Métrica L2", fontsize=12)
axes[1, 0].set_ylabel("Métrica Coseno", fontsize=12)

# Mostrar Ancla en la primera columna
for i in range(2):
    axes[i, 0].imshow(images[idx_ancla], cmap='gray')
    axes[i, 0].set_title(f"Ancla (Sujeto {y[idx_ancla]})")
    axes[i, 0].set_xticks([])
    axes[i, 0].set_yticks([])

# Mostrar Top 3 para L2
for i, idx in enumerate(top3_l2_idx):
    axes[0, i+1].imshow(images[idx], cmap='gray')
    axes[0, i+1].set_title(f"Sujeto {y[idx]}\nDist: {distancias_l2[idx]:.2f}")
    axes[0, i+1].set_xticks([])
    axes[0, i+1].set_yticks([])

# Mostrar Top 3 para Coseno
for i, idx in enumerate(top3_coseno_idx):
    axes[1, i+1].imshow(images[idx], cmap='gray')
    axes[1, i+1].set_title(f"Sujeto {y[idx]}\nSimilitud: {similitud_coseno[idx]:.4f}")
    axes[1, i+1].set_xticks([])
    axes[1, i+1].set_yticks([])

plt.tight_layout()
plt.show()
```



(c) Discusión: ¿Las imágenes más cercanas corresponden a la misma persona? ¿Por qué la distancia Euclidiana directa sobre píxeles es estadísticamente frágil si la persona mueve ligeramente la cabeza o hay un cambio global en la iluminación?

Al observar los resultados para nuestra imagen ancla, evidenciamos que las 3 imágenes más cercanas obtenidas mediante la distancia Euclidiana (L_2) y la similitud del Coseno coinciden exactamente, tanto en las imágenes seleccionadas como en su orden. Además, las tres corresponden efectivamente a la misma persona del ancla. Para esta ancla en particular, dimos con una persona cuyas fotos no cambiaron mucho de posición ni de brillo por lo cual ambas medidas no tuvieron mayor problema en identificar correctamente las imagenes más cercanas. De acuerdo con la librería, muchas imagenes de Olivetti presenta condiciones sumamente controladas: los rostros están previamente alineados, centrados y sobre un fondo homogéneo, lo que favorece temporalmente la comparación directa de píxeles en $\mathbb{R}^{4096}$.

No obstante, calcular la distancia L_2 directamente sobre los píxeles crudos es un enfoque estadísticamente frágil en sistemas biométricos reales debido a dos factores matemáticos:

- Sensibilidad extrema al movimiento (Desalineación espacial): Si la persona mueve ligeramente la cabeza o cambia su postura, el píxel x_i que antes registraba la pupila oscura ahora se comparará con un píxel y_i correspondiente a la piel clara. Dado que la métrica calcula $d_{L2}(x, y) = \sqrt{\sum_{i=1}^{4096}(x_i - y_i)^2}$, estas diferencias al cuadrado penalizan drásticamente el resultado. El algoritmo no comprende la estructura facial global; solo percibe un desajuste numérico masivo, disparando la distancia global.

- Sensibilidad a cambios de iluminación (Desplazamiento de magnitud): la distancia Euclidiana mide la magnitud pura entre dos puntos en el espacio. Si a la misma persona se le toma una fotografía idéntica pero con mayor iluminación global, el valor de los píxeles aumentará por un factor o constante c. Al comparar la imagen original x con la iluminada $y = x + c$, la distancia sería

$d_{L2}(x, x+c) = \sqrt{\sum_{i=1}^{4096}(-c)^2} = 64c$. Aunque la topología del rostro sea exactamente la misma, la métrica los clasificará como entidades muy lejanas en el espacio, fallando en reconocer la identidad.

Tarea 2: Factorización Matricial y Eigenfaces

Extraeremos las direcciones de máxima varianza para reducir drásticamente la dimensionalidad.

(a) Centren la matriz de datos X restando el vector de la media global (el "rostro promedio").

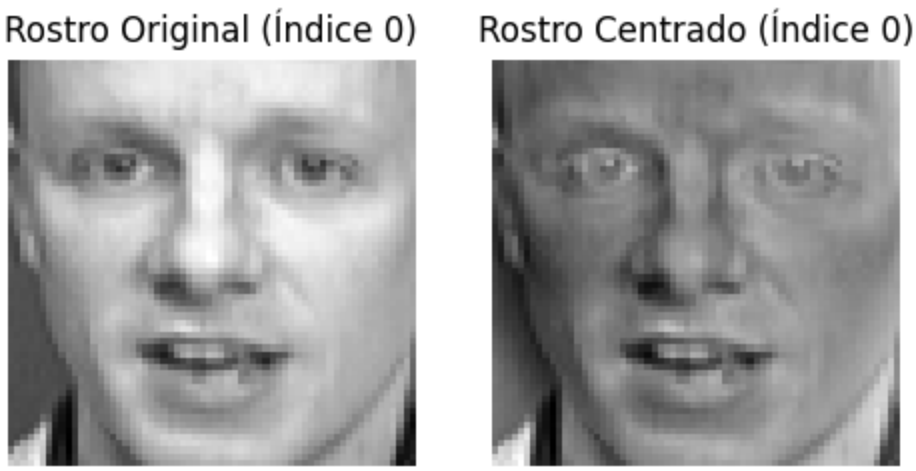
In [67]:
```
# --- CENTRAR LOS DATOS ---
# Calcular el rostro promedio (media a lo largo de las columnas/características -> media de cada
rostro_promedio = np.mean(X, axis=0)

# Centramos la matriz restando este promedio a cada imagen
X_centrada = X - rostro_promedio

# Verificamos las dimensiones para asegurar que no cambiaron
print(f"Dimensiones de la matriz de rostro promedio: {rostro_promedio.shape}") # Debería ser (40
print()
print(f"Dimensiones de X original: {X.shape}")
print(f"Dimensiones de X centrada: {X_centrada.shape}")

# --- Visualización del Rostro Promedio ---
plt.figure(figsize=(3, 3))
# Recordamos redimensionar el vector de 4096 a 64x64 para poder graficarlo
plt.imshow(rostro_promedio.reshape(64, 64), cmap='gray')
plt.title("Rostro Promedio (Media Global)")
plt.axis('off')
plt.show()
```

```
Dimensiones de la matriz de rostro promedio: (4096,)

Dimensiones de X original: (400, 4096)
Dimensiones de X centrada: (400, 4096)
```

Rostro Promedio (Media Global)

También podemos observar como sería el rostro centrado de nuestro rostro ancla:

In [68]:
```
# --- Visualización de un Rostro Original vs. su versión Centrada ---

# Elegimos a qué persona del dataset queremos mirar (índice del 0 al 399)
indice = 0

# Extraemos el vector original y el centrado
rostro_original = X[indice]
rostro_centrado = X_centrada[indice]

# Creamos la figura comparativa
fig, axes = plt.subplots(1, 2, figsize=(5, 2.5))

# 1. Gráfica de la imagen original
axes[0].imshow(rostro_original.reshape(64, 64), cmap='gray')
axes[0].set_title(f"Rostro Original (Índice {indice})")
axes[0].axis('off')

# 2. Gráfica de la imagen centrada
axes[1].imshow(rostro_centrado.reshape(64, 64), cmap='gray')
axes[1].set_title(f"Rostro Centrado (Índice {indice})")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```



(b) Utilicen la Descomposición en Valores Singulares (SVD) Reducida para encontrar los componentes principales.

En álgebra lineal, la SVD descompone nuestra matriz X en el producto de tres matrices:

$$X = U\Sigma V^T$$

U: Matriz de vectores singulares izquierdos.

Σ (en código suele llamarse S): Matriz diagonal con los valores singulares.

V^T: Matriz de vectores singulares derechos transpuestos (nuestros famosos componentes principales).

Dado que nuestra matriz X tiene dimensiones 400×4096, el rango máximo de la matriz es 400. Si calculáramos la SVD completa, V^T sería una matriz gigantesca de 4096×4096, donde la inmensa mayoría de las filas estarían vacías (dimensiones nulas). Al usar la versión "reducida", le decimos al algoritmo que solo calcule los primeros 400 componentes útiles.

In [69]:
```
# --- SVD (Singular Vector Decomposition) REDUCIDA ---
# El parámetro full_matrices=False nos asegura calcular la SVD reducida,
# evitando computar dimensiones nulas ineficientes.
# NO usamos librerías PCA preconstruidas.

print("Calculando SVD...")
U, S, Vt = np.linalg.svd(X_centrada, full_matrices=False)

print(f"Dimensiones de U: {U.shape}")
# S se devuelve como un vector 1D con los valores de la diagonal para ahorrar memoria
print(f"Dimensiones de S (Valores Singulares): {S.shape}")
print(f"Dimensiones de V transpuesta (Vt): {Vt.shape}")
```

```
Calculando SVD...
Dimensiones de U: (400, 400)
Dimensiones de S (Valores Singulares): (400,)
Dimensiones de V transpuesta (Vt): (400, 4096)
```

(c) ¿Qué es un Eigenface? Es simplemente un vector propio (una fila de la matriz VT) que ha sido re-dimensionado a su forma original de imagen de 64 × 64. Estos representan los “bloques de construcción” visuales de los rostros. Seleccionen las primeras 5 filas de VT, reformateen estos vectores a imágenes y visualícenlas en una cuadrícula de 1 fila y 5 columnas.¿Qué características capturan?

In [70]:
```
# --- EIGENFACES ---
# Un Eigenface es una fila de Vt, redimensionada a 64x64.
# Seleccionamos las primeras 5 filas (las de mayor varianza)

eigenfaces = Vt[:5]

fig, axes = plt.subplots(1, 5, figsize=(15, 3))
for i in range(5):
    # Redimensionamos el vector de 4096 a una matriz de 64x64
    eigenface_img = eigenfaces[i].reshape(64, 64)
    axes[i].imshow(eigenface_img, cmap='gray')
    axes[i].set_title(f"Eigenface {i+1}")
    axes[i].axis('off')

fig.suptitle("Primeros 5 Eigenfaces (Vectores Propios de Máxima Varianza)", fontsize=16)
plt.show()
```

¿Qué características capturan los Eigenfaces?

Matemáticamente, estos Eigenfaces son los vectores propios de la matriz de covarianza de los datos. Actúan como una base ortogonal; es decir, son los "bloques de construcción" fundamentales con los que se puede reconstruir cualquier rostro del dataset mediante una combinación lineal.

Al visualizar esto, vemos rostros "fantasmales". El Eigenface 1 (el primer componente principal) captura la fuente de mayor variación en todo el conjunto de datos. En este caso la iluminación. Los demás eigenfaces capturan estructuras geométricas predominantes, como la forma de la mandíbula, las cuencas de los ojos o el contorno de la nariz. ¡Son la "base matemática" con la que se puede armar cualquier rostro!

(d) Multipliquen los datos centrados por los primeros $k = 50$ componentes de V^T para proyectar las imágenes originales a un nuevo espacio latente. Su nueva matriz de características será $Z \in R^{400\times50}$.

Aquí aplicamos la reducción de dimensionalidad.

Para proyectar nuestros datos, no necesitamos las 400 dimensiones; el taller nos pide truncar esta matriz y quedarnos solo con los primeros $k = 50$ componentes.

- Tomamos las primeras 50 filas de V^T.
- Transponemos ese resultado para obtener una matriz V_k de dimensiones 4096×50.
- Finalmente, realizamos el producto matricial o producto punto entre nuestra matriz de datos centrada (400×4096) y nuestra matriz de componentes V_k (4096×50).

Por las reglas del álgebra lineal multivariada, el resultado será nuestra nueva matriz Z de dimensiones 400×50

In [71]:
```
# --- PROYECCIÓN AL ESPACIO LATENTE ---
k = 50

# Extraer las primeras 50 filas de Vt y transponer para obtener V_k
# Vt[:k, :] toma las filas de 0 a 49 y todas las columnas.
# .T transpone el resultado para que quede de (4096, 50)
V_k = Vt[:k, :].T

# Proyectar los datos centrados al nuevo espacio latente Z mediante producto matricial
Z = np.dot(X_centrada, V_k)

# Comprobación de dimensiones para el informe
print(f"Dimensiones de X_centrada: {X_centrada.shape}")
print(f"Dimensiones de V_k (primeros {k} componentes): {V_k.shape}")
print(f"Dimensiones de la nueva matriz latente Z: {Z.shape}") # Debería ser (400, 50)
```

```
Dimensiones de X_centrada: (400, 4096)
Dimensiones de V_k (primeros 50 componentes): (4096, 50)
Dimensiones de la nueva matriz latente Z: (400, 50)
```

> **Para comprender mejor la relación de compresión y pérdida de información del proceso que hemos desarrollado hasta este punto realizaremos dos pasos adicionales:**

- (i) medir matemáticamente cuánta información ("varianza") retienen esos 50 componentes.
- (ii) realizaremos el proceso inverso para reconstruir nuestra imagen "Ancla" a partir de esos 50 componentes y la compararemos visualmente con la original.

In [72]:
```
# --- PASO EXTRA: Varianza Explicada y Reconstrucción ---

# 1. Calcular la varianza explicada por los primeros k=50 componentes
# En la SVD, la varianza es proporcional al cuadrado de los valores singulares (S)
varianza_total = np.sum(S**2)
varianza_k50 = np.sum(S[:k]**2)

porcentaje_varianza = (varianza_k50 / varianza_total) * 100
print(f"Los primeros {k} componentes explican el {porcentaje_varianza:.2f}% de la varianza total

# 2. Reconstruir la imagen "Ancla" (índice 0) a partir de sus 50 coordenadas en Z
# Tomamos el vector del ancla en nuestro nuevo espacio latente Z (dimensiones: 50,)
z_ancla = Z[0]

# Hacemos la operación matemática inversa:
# Multiplicamos por la matriz de componentes (Vk) y sumamos el rostro promedio que habíamos resta
ancla_reconstruida = np.dot(z_ancla, V_k.T) + rostro_promedio

# 3. Visualizar y comparar
fig, axes = plt.subplots(1, 2, figsize=(6, 3))
fig.suptitle(f"Reconstrucción del Ancla usando solo {k} componentes", fontsize=14)

# Ancla Original (4096 píxeles reales)
axes[0].imshow(X[0].reshape(64, 64), cmap='gray')
axes[0].set_title("Ancla Original (100% de la info)")
axes[0].axis('off')

# Ancla Reconstruida (Desde 50 componentes)
axes[1].imshow(ancla_reconstruida.reshape(64, 64), cmap='gray')
axes[1].set_title(f"Reconstruida ({porcentaje_varianza:.1f}% de la info)")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

```
Los primeros 50 componentes explican el 87.38% de la varianza total.
```

El análisis nos indica que los primeros 50 componentes explican el 87.38% de la varianza total. Esto es un resultado espectacular del álgebra lineal ya que logramos reducir el tamaño de los datos en más de un 98% (de 4096 variables pasamos a solo 50), pero logramos retener la inmensa mayoría de la información estructural del rostro.

Al reconstruir la imagen ancla usando solo 50 componentes principales notamos un muy buen desempeño considerando el cambio de dimensionalidad. Es bastante parecida a la original (inclinación, gesto facial, formas...), pero más "suavizada" y borrosa, perdiendo detalles finos. Esto ocurre porque la SVD actuó como un filtro de ruido. Al desechar los otros 350 componentes que capturaban el 10% restante de la varianza, matemáticamente eliminamos variaciones y detalles (como ruido estático de la cámara, arrugas muy diminutas o cambios ínfimos de luz).

Este paso extra es la demostración visual y matemática de por qué nuestro espacio latente $Z \in \mathbb{R}^{400\times 50}$ es tan poderoso, especialmente pensando en la clasificación que haremos en la Tarea 4 de este taller.

Tarea 3: Detección de Anomalías Sensible a la Correlación

Verificaremos si el espacio latente es capaz de aislar datos atípicos usando estadística multivariada.

La detección de las anomalías o intruso entre las fotos vamos a usar la **Distancia de Mahalanobis**, debido a que esta distancia, a diferencia de la Distancia Eclidiana, como fue evaluado en los puntos anteriores, tiene en cuenta la varianza y la correlación entre las dimensiones del espacio latente.

Entrenado en detalle, el problema de usar la distancia Euclidiana en el espacio latente es que, aunque nuestras 50 características (Eigenfaces) son ortogonales, no tienen la misma "importancia". El Componente 1 captura muchísima más varianza natural que el Componente 50. La distancia Euclidiana asume que todas las dimensiones tienen la misma escala y forma una "esfera" perfecta alrededor del centroide. Si un rostro varía mucho en el Componente 1 (lo cual es normal, ej. cambios de luz) pero el intruso varía un poco en el Componente 50 (lo cual es estadísticamente imposible para un rostro humano), la distancia Euclidiana podría darles el mismo valor de "rareza", fallando en detectar la anomalía.

Por otro lado, Mahalanobis brilla pues, al incluir la matriz de covarianza inversa (Σ^{-1}) en la ecuación matemática, estandariza la varianza de cada eje y modela los datos como una "elipse" (un hiperelipsoide). Matemáticamente, penaliza fuertemente cualquier desviación en direcciones donde la varianza natural es baja (como los últimos componentes).

(a) Tomen la “imagen intrusa” descrita en la sección de datos, réstenle el rostro promedio y proyéctenla utilizando la misma transformación del paso anterior para obtener un vector intruso $z_{intruso} \in R^{50}$. Añádanlo a su matriz Z

Se crea y proyecta la imagen intrusa. Para la creación de la imagen intrusa: la primera mitad del vector es `1.0` (blanco) y la segunda mitad `0.0` (negro). Luego la centramos con el mismo `rostro_promedio` de la Tarea 2 y la proyectamos al espacio latente con la misma transformación `V_k`.

In [73]:
```
# Creamos la imagen intrusa: primera mitad = 1.0 (blanco), segunda mitad = 0.0 (negro)
imagen_intrusa = np.zeros(4096)
imagen_intrusa[:2048] = 1.0

# Visualizamos la imagen intrusa
fig, ax = plt.subplots(1, 1, figsize=(3, 3))
ax.imshow(imagen_intrusa.reshape(64, 64), cmap='gray')
ax.set_title('Imagen Intrusa (mitad blanca / mitad negra)')
ax.axis('off')
plt.tight_layout()
plt.show()

# Centramos la imagen intrusa con el mismo rostro_promedio de la Tarea 2
intrusa_centrada = imagen_intrusa - rostro_promedio

# Proyectamos al espacio latente usando V_k (4096 x 50)
z_intruso = intrusa_centrada @ V_k   # vector de R^50
print(f'Forma del vector intruso proyectado: {z_intruso.shape}')

# Añadimos el vector intruso a Z -> Z_ext de dimension (401, 50)
Z_ext = np.vstack([Z, z_intruso])
print(f'Dimension de Z_ext (con intruso): {Z_ext.shape}')
```

Imagen Intrusa (mitad blanca / mitad negra)

```
Forma del vector intruso proyectado: (50,)
Dimension de Z_ext (con intruso): (401, 50)
```

(b) En este espacio de 50 dimensiones, calculen la Distancia de Mahalanobis de todas las imágenes respecto al centroide global del dataset1. Recuerden que este cálculo requiere estimar e invertir la matriz de covarianza de Z.

En este literal vamos a hallar la **Distancia de Mahalanobis** al centroide global que es el vector de medías multivariado como lo indica la nota en el enunciado. Lo que quiere decir, que no solo se busca medir la distancia del centroide al punto determinado, sino que también se mide que tan diferente es ese punto del resto de puntos.

Entonces, se tiene que la Distancia de Mahalanobis para una observación $\mathbf{z}_i$ respecto al centroide global $\bar{\mathbf{z}}$ se define como:

$$D_i^2 = (\mathbf{z}_i - \bar{\mathbf{z}})^\top \mathbf{S}^{-1} (\mathbf{z}_i - \bar{\mathbf{z}})$$

donde $\mathbf{S}$ es la matriz de covarianza de $Z_{ext} \in \mathbb{R}^{401\times 50}$, que son la 400 imagenes reales y la imagen intrusa. Como se vio en clase, esta métrica es un **producto interno ponderado** $\langle x, y\rangle_{S^{-1}} = x^\top S^{-1} y$ que normaliza cada dirección por su varianza y descorrelaciona las dimensiones, logrando una medida de distancia estadística genuina.

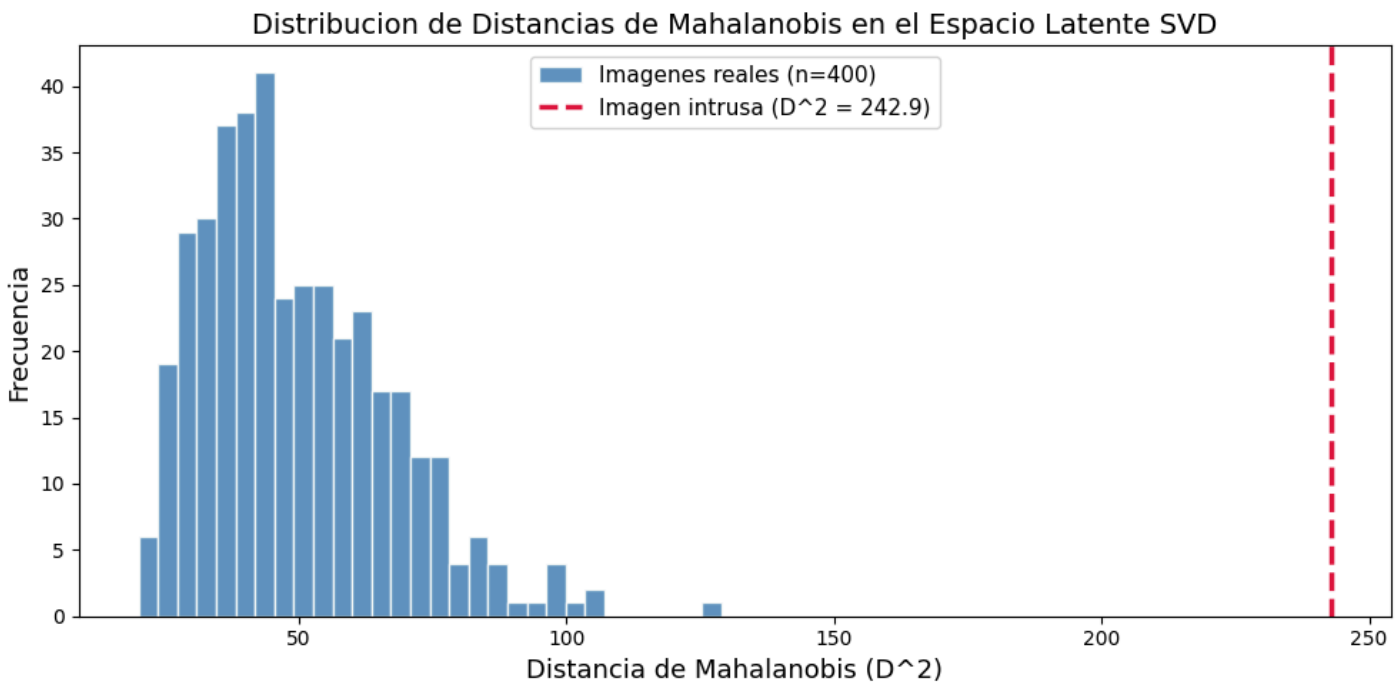
In [74]:
```python
# Centroide global del espacio latente (media por dimension)
centroide = np.mean(Z_ext, axis=0)   # Vector de R^50
print(f'Forma del centroide: {centroide.shape}')

# Matriz de covarianza de Z_ext (50 x 50)
# np.cov espera variables en filas, por eso transponemos
S = np.cov(Z_ext.T)
print(f'Forma de la matriz de covarianza S: {S.shape}')

# Invertimos la matriz de covarianza
S_inv = np.linalg.inv(S)

# Calculamos D^2_i para cada imagen (las 401 filas)
N_ext = Z_ext.shape[0]
distancias_maha = np.zeros(N_ext)

for i in range(N_ext):
    diff = Z_ext[i] - centroide
    distancias_maha[i] = diff @ S_inv @ diff

print(f'\nDistancia Mahalanobis de la imagen intrusa (indice 400): {distancias_maha[400]:.2f}')
print(f'Distancia maxima de las imagenes reales (0-399): {distancias_maha[:400].max():.2f}')
print(f'Distancia media de las imagenes reales: {distancias_maha[:400].mean():.2f}')
```

```
Forma del centroide: (50,)
Forma de la matriz de covarianza S: (50, 50)

Distancia Mahalanobis de la imagen intrusa (indice 400): 242.90
Distancia maxima de las imagenes reales (0-399): 128.83
Distancia media de las imagenes reales: 49.39
```

(c) Grafiquen un histograma de las distancias calculadas y marquen con una línea vertical roja u otro color llamativo dónde quedó ubicada la imagen intrusa.

Se realiza un histograma para visualizar las ditancias entre de las imagenes reales y la intrusa, para verificar lo que detectamos matemáticamente.

In [75]:
```
fig, ax = plt.subplots(figsize=(10, 5))

# Histograma de las 400 imagenes reales
ax.hist(distancias_maha[:400], bins=30, color='steelblue', edgecolor='white',
        alpha=0.85, label='Imagenes reales (n=400)')

# Linea vertical para la imagen intrusa
ax.axvline(distancias_maha[400], color='crimson', linewidth=2.5, linestyle='--',
           label=f'Imagen intrusa (D^2 = {distancias_maha[400]:.1f})')

ax.set_xlabel('Distancia de Mahalanobis (D^2)', fontsize=13)
ax.set_ylabel('Frecuencia', fontsize=13)
ax.set_title('Distribucion de Distancias de Mahalanobis en el Espacio Latente SVD', fontsize=14)
ax.legend(fontsize=11)
plt.tight_layout()
plt.show()

print(f'Percentil 99 de imagenes reales: {np.percentile(distancias_maha[:400], 99):.2f}')
print(f'La intrusa supera al {(distancias_maha[400] > distancias_maha[:400]).mean()*100:.1f}% de
```




```
Percentil 99 de imagenes reales: 99.16
La intrusa supera al 100.0% de las imagenes reales.
```

(d) Discusión: Dado que las características en el espacio latente SVD son ortogonales, analicen matemáticamente por qué la métrica de Mahalanobis, al incluir la matriz de covarianza, es superior a una simple distancia Euclidiana para confirmar que esa imagen es un outlier

Dado que las características en el espacio latente SVD (Singular Value Decomposition) son ortogonales por construcción (las columnas de V_k forman una base ortonormal), las componentes de Z son **incorreladas entre sí**. Esto implica que la matriz de covarianza $\mathbf{S}$ es aproximadamente diagonal:

$$\mathbf{S} \approx \mathrm{diag}(\sigma_1^2, \sigma_2^2, \ldots, \sigma_{50}^2)$$

donde $\sigma_1^2 \gg \sigma_2^2 \gg \cdots \gg \sigma_{50}^2$, pues los valores singulares de la SVD van decreciendo en orden.

Como $\mathbf{S}$ es diagonal, su inversa también lo es:

$$\mathbf{S}^{-1} \approx \mathrm{diag}(1/\sigma_1^2, 1/\sigma_2^2, \ldots, 1/\sigma_{50}^2)$$

Con esto, la distancia de Mahalanobis se simplifica a:

$$D_i^2 = \sum_{j=1}^{50} \frac{(z_{ij} - \bar{z}_j)^2}{\sigma_j^2}$$

Comparándola directamente con la Euclidiana:

$$d_E^2 = \sum_{j=1}^{50} (z_{ij} - \bar{z}_j)^2$$

La diferencia estructural es el factor $1/\sigma_j^2$: Mahalanobis **pondera cada dimensión por el inverso de su varianza**. Las últimas componentes tienen σ_j^2 muy pequeño (casi no varían entre rostros reales), por lo que $1/\sigma_j^2$ es muy grande. La imagen intrusa, al ser estructuralmente incompatible con cualquier cara humana, presenta desviaciones significativas precisamente en esas componentes de baja varianza, lo que dispara su D^2 a 242.90 — superando en casi el doble a la imagen real más alejada (128.83).

La distancia Euclidiana, al tratar todas las dimensiones por igual, subestimaría sistemáticamente la rareza estadística de la intrusa en esas componentes. Por eso Mahalanobis no mide distancia geométrica sino **rareza estadística**, y es la métrica correcta para este problema.

Tarea 4: Optimización Convexa: Clasificación Binaria

El objetivo final es demostrar que nuestra reducción a 50 dimensiones retuvo la información esencial que permite distinguir el rostro de una persona del de otra. Para ello, entrenaremos un clasificador matemático y validaremos su poder predictivo.

Preparación y Partición de los Datos:

In [76]:
```python
from sklearn.model_selection import train_test_split

# --- 4. Preparación y Partición de los Datos ---

# 1. Seleccionar dos sujetos (ej. Sujeto 0 y Sujeto 1)
sujeto_A = 0
sujeto_B = 1

# 2. Filtrar la matriz Z (la de 400x50, asegurándonos de NO incluir al intruso)
# Las etiquetas originales 'y' tienen el mismo orden que las filas de Z
mascara_sujetos = (y == sujeto_A) | (y == sujeto_B)

# Aplicamos la máscara para quedarnos solo con las 20 filas correspondientes (10 de cada uno)
Z_filtrado = Z[mascara_sujetos]
y_filtrado = y[mascara_sujetos]

# 3. Transformar las etiquetas al formato {-1, 1} para la optimización matemática
# Usamos np.where: Si la etiqueta es el sujeto_A, le asignamos 1; de lo contrario, -1
y_binario = np.where(y_filtrado == sujeto_A, 1, -1)

# 4. Partición (Train/Test)
# Dividimos las 20 muestras: 16 para entrenamiento (80%) y 4 para prueba (20%)
# random_state=42 asegura que si vuelves a correr el código, te haga la misma partición
Z_train, Z_test, y_train, y_test = train_test_split(
    Z_filtrado, y_binario, test_size=0.2, random_state=42, stratify=y_binario
    # El parámetro stratify obliga a la función a mantener la misma proporción original:
    # garantiza que en Z_train haya exactamente 8 imágenes del Sujeto 0 y 8 del Sujeto 1.
    # Por qué hacerlo?  Al tener tan pocos datos (solo 20 imágenes), si hacemos un corte completa
    # podríamos tener la mala suerte de que en las 16 imágenes de entrenamiento queden 10 del Su
)

# Imprimir las dimensiones para el informe
print(f"Dimensiones de Z_train (N muestras, k características): {Z_train.shape}")
print(f"Dimensiones de y_train: {y_train.shape}")
print(f"Dimensiones de Z_test: {Z_test.shape}")
print(f"Dimensiones de y_test: {y_test.shape}")
print(f"\nEtiquetas de entrenamiento (y_train): {y_train}")
```

```
Dimensiones de Z_train (N muestras, k características): (16, 50)
Dimensiones de y_train: (16,)
Dimensiones de Z_test: (4, 50)
Dimensiones de y_test: (4,)

Etiquetas de entrenamiento (y_train): [-1 -1  1  1  1  1  1 -1 -1  1  1 -1  1 -1 -1 -1]
```

(a) Derivación Analítica: Calculen e incluyan en su reporte el vector gradiente ∇J(w), aplicando cuidadosamente la regla de la cadena sobre la suma.

Partimos de la función de costo objetivo:

$$J(w) = \underbrace{\frac{1}{2N}\sum_{i=1}^{N}(w^T z_i - y_i)^2}_{\text{Término de Error}} + \underbrace{\frac{\lambda}{2}\sum_{j=1}^{k} w_j^2}_{\text{Término de Regularización}}$$

Donde:

1. El Término de Error: en terminos generales mide qué tan equivocadas están las predicciones del modelo frente a la realidad de los datos de entrenamiento. Siendo:

- w (El vector de pesos): es un vector de k números (en nuestro caso, 50, uno para cada Eigenface). El objetivo de todo este proceso de optimización es encontrar los valores perfectos para este vector. Es decir, nos interesa conocer el valor de w que asigne el peso o importancia optimo a cada Eigenface de tal manera que minimice el error al intentar predecir la etiqueta de los rostros evaluados.

- z_i (Los datos de entrada): Es la representación en el espacio latente de la imagen número i. Son los 50 números que extrajimos con la SVD en la Tarea 2.

- $w^T z_i$ (La predicción): Es el producto punto entre los pesos y las características de la imagen. El modelo multiplica cada característica por su peso correspondiente y suma todo para dar un puntaje. Si el puntaje es positivo, cree que es el Sujeto 0; si es negativo, cree que es el Sujeto 1.

  En cada iteración, el algoritmo mira una imagen, calcula su error y usa el gradiente (nuestra fórmula matemática) como una brújula. El gradiente le dice a cada uno de los 50 pesos que pasa si aumenta su magnitud. Ajustando su magnitud y signo iterativamente encontramos el vector óptimo w^* que minimiza el error de la predicción.

- y_i (La etiqueta real): Es la respuesta correcta (1 o -1) que nosotros le dimos en el paso de preparación.

- $(w^T z_i - y_i)^2$ (El error al cuadrado): Calcula la diferencia entre lo que predijo el modelo y la realidad, y lo eleva al cuadrado. Se eleva al cuadrado por dos razones: para que los errores negativos y positivos no se cancelen entre sí, y para castigar mucho más fuerte a los errores grandes que a los pequeños.

- $\frac{1}{2N} \sum_{i=1}^{N}$ (El promedio): Suma el error de las N imágenes de entrenamiento (nuestras 16 imágenes) y lo divide entre N para sacar un promedio. El número 2 en el divisor ($\frac{1}{2N}$) es un simple truco matemático para que, al derivar y bajar el exponente 2, se cancelen y la fórmula del gradiente quede más limpia.

2. El Término de Regularización: es la penalización de Ridge (o regularización L_2) y evita que el modelo se "aprenda de memoria" los datos (sobreajuste). Siendo:

- w_j^2 (El tamaño de los pesos): Toma cada uno de los 50 pesos individuales que el modelo está aprendiendo y los eleva al cuadrado.

- $\sum_{j=1}^{k} w_j^2$ (La penalización total): Suma todos esos pesos al cuadrado. Si el modelo intenta poner números gigantescos en el vector w para memorizar el ruido de las imágenes, esta sumatoria se vuelve enorme. Como el modelo quiere minimizar $J(w)$, esta parte lo obliga a mantener los pesos lo más pequeños y cercanos a cero posible.

- λ: este hiperparametro nos permite definir qué tanta importancia le daremos a mantener los pesos pequeños frente a cometer errores de predicción.

  - Si λ es 0, no hay regularización, por lo que el modelo podría asignar valores w muy altos para clasificar más fácilmente las imegenes pero con el riesgo de darle importancia a características inútiles y hacer overfitting (memorizando detalles), con lo que perdería capacidad de generalización para predecir adecuadamente nuevas imagenes.

- Si λ es muy alto, el modelo preferirá tener pesos en cero aunque se equivoque en todas las predicciones. Ya que por defenición matematica de la función matemática, con un λ muy alto cualquier pequeño valor de $w*$ escalara rapidamente el error total.
- El divisor 2: Al igual que en el término de error, el $\frac{\lambda}{2}$ está ahí para que el 2 se cancele al derivar.

Conociendo la función de costo, y dado que el operador gradiente ∇_w es lineal, calculamos la derivada de la suma como la suma de los gradientes individuales:

1. Gradiente del término de Error (Aplicando la Regla de la Cadena Multivariada):

Definimos el término de error para la i-ésima muestra como la composición de dos funciones $h(w) = f(g(w))$, donde:

- Función interna $g : \mathbb{R}^k \to \mathbb{R}$: $g(w) = w^T z_i - y_i$
- Función externa $f : \mathbb{R} \to \mathbb{R}$: $f(u) = u^2$

Por la regla de la cadena para funciones vectoriales, el gradiente es el producto de la transpuesta del Jacobiano de la función interna por el gradiente de la función externa:

$$\nabla_w h(w) = J_g(w)^T \nabla f(g(w))$$

Calculamos ambas partes:

La derivada de la función externa respecto a su argumento u es $\nabla f(u) = 2u = 2(w^T z_i - y_i)$.

El Jacobiano de la función interna (lineal en w) es $J_g(w) = z_i^T$.

Al trasponerlo, obtenemos el vector columna z_i.

Multiplicando, obtenemos el gradiente para la muestra i:

$$\nabla_w (w^T z_i - y_i)^2 = z_i \cdot 2(w^T z_i - y_i) = 2(w^T z_i - y_i) z_i$$

Sustituyendo esto en la sumatoria original y multiplicando por la constante $\frac{1}{2N}$, los factores 2 se cancelan:

$$\nabla_w (\text{Error}) = \frac{1}{N} \sum_{i=1}^{N} (w^T z_i - y_i) z_i$$

2. Gradiente del término de Regularización (L_2):

El término de penalización es $R(w) = \frac{\lambda}{2} \sum_{j=1}^{k} w_j^2$.

Esta sumatoria equivale a la forma cuadrática $R(w) = \frac{\lambda}{2}(w^T w)$.

Al aplicar el operador gradiente estándar sobre la norma cuadrada de un vector, obtenemos:

$$\nabla_w R(w) = \frac{\lambda}{2}(2w) = \lambda w$$

3. Vector Gradiente Final:

Sumando los resultados obtenidos, el vector gradiente analítico que define la dirección de máximo crecimiento de la función de costo es:

$$\nabla J(w) = \frac{1}{N}\sum_{i=1}^{N}(w^T z_i - y_i)z_i + \lambda w$$

(b) Implementación (SGD):

Programen en Python el algoritmo de Gradiente Descendente Estocástico. Iteren únicamente sobre las N muestras de entrenamiento para encontrar los pesos óptimos $w*$. Demuestren la convergencia generando una gráfica de la función de Costo vs. Número de Iteraciones.

Ya conocemos la función de costo y el vector gradiente que nos indica la dirección de máximo crecimiento. Dado que los valores de las características z_i, las etiquetas y_i y el parámetro de regularización λ son fijos, la única variable que podemos ajustar para minimizar el error son los pesos w asociados a nuestras Eigenfaces (inicialmente mantendremos $\lambda = 0.1$).

Es aquí donde entra el algoritmo de Gradiente Descendente Estocástico (SGD), el cual nos permitirá hallar el vector de pesos óptimo w^* que minimice la función de costo. A diferencia del método tradicional por lotes (Batch) que procesa todo el conjunto de datos a la vez, el SGD actualiza el vector de pesos evaluando muestra por muestra.

El procedimiento analítico que seguiremos es el siguiente:

- Inicialización: Comenzamos definiendo un vector de pesos w compuesto netamente por ceros.
- Cálculo del Gradiente Estocástico: Para cada imagen individual z_i y su etiqueta y_i, calculamos la dirección de máximo crecimiento del error utilizando el gradiente derivado analíticamente:

$$\nabla J_i(w) = (w^T z_i - y_i)z_i + \frac{\lambda}{N}w$$

- Actualización (El paso de descenso): Ajustamos los pesos w moviéndonos en la dirección contraria al gradiente, controlando el tamaño del paso con una tasa de aprendizaje (α):

$$w_{nuevo} = w_{viejo} - \alpha \nabla J_i(w)$$

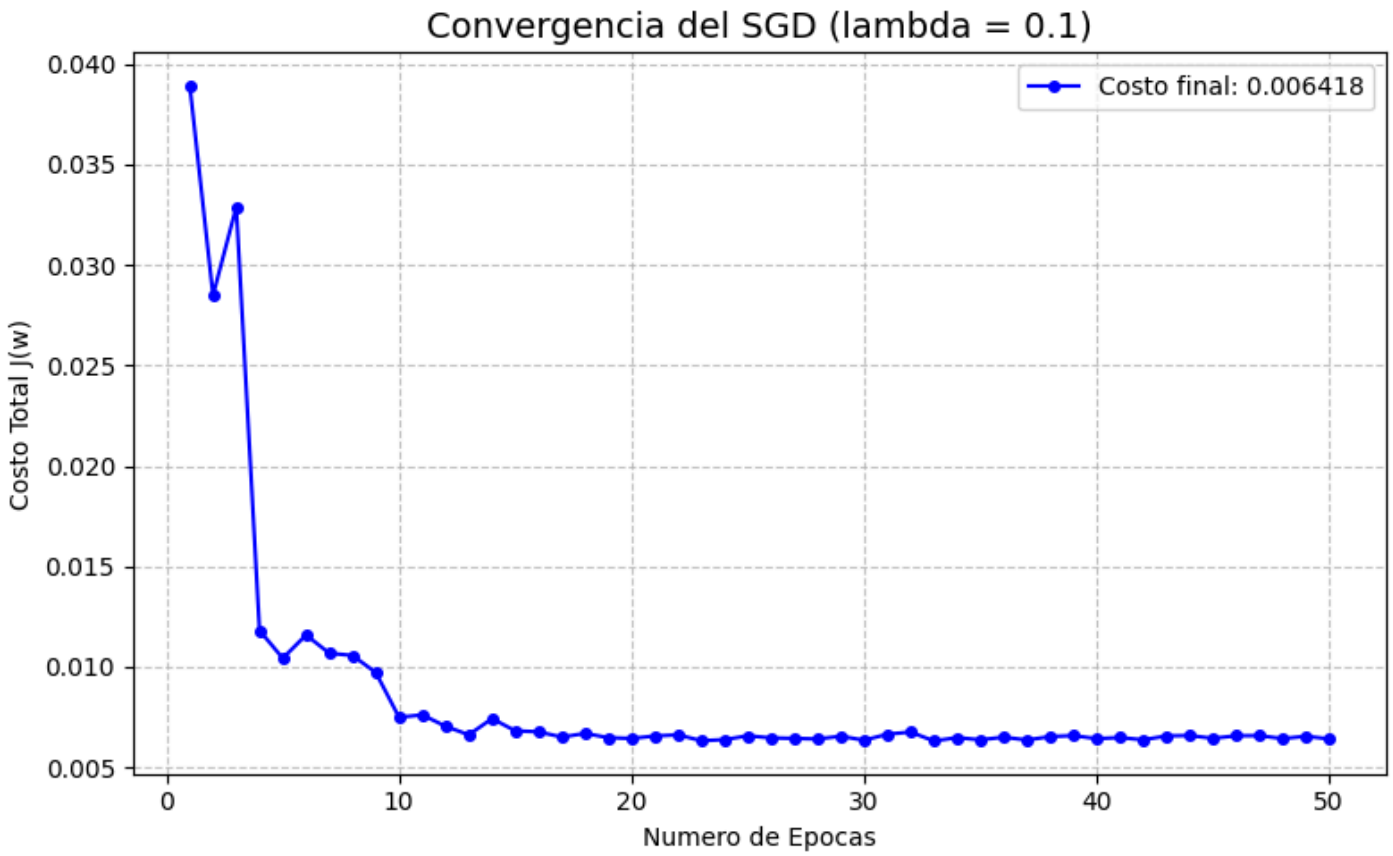
- Iteración por muestra: Inicialmente, este proceso supondrá 16 iteraciones (una por cada imagen del conjunto de entrenamiento). Para la primera imagen (seleccionada aleatoriamente), partimos del vector w en ceros. Sin embargo, para las siguientes iteraciones, calcularemos el gradiente partiendo del vector w hallado en la imagen inmediatamente anterior. ¿Por qué iterar sobre una imagen distinta y empezar desde el w previo? Porque cada imagen, dados sus valores particulares de z_i, genera su propia función topográfica de error y requiere pesos particulares para minimizarlo. Iterar continuamente sobre imágenes distintas, arrastrando el aprendizaje anterior, evita que el modelo memorice una sola foto y le permite hallar un w^* de consenso que considere las necesidades conjuntas de todas las imágenes.
- El ciclo de Épocas: Como es muy probable que un único ciclo de 16 iteraciones no sea suficiente para hallar el w^* que minimice el error global, repetiremos este proceso durante N pasadas por el dataset

completo (épocas). Al pasar de una época a otra, siempre partiremos del último w hallado para garantizar un aprendizaje acumulativo. Además, al inicio de cada época, mezclaremos aleatoriamente el orden de las imágenes para evitar oscilaciones sesgadas en el descenso.

In [80]:
```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Parámetros del Algoritmo ---
epochs = 50  #definimos un valor pequeño de 50 inicialmente esperando que al ser solo 16 imagenes
learning_rate = 0.01        #punto de partida normalmente utilziado o standar
N, k = Z_train.shape
lambda_base = 0.1           # Valor sugerido inicialmente

# --- 2. Inicialización ---
w_optimo = np.zeros(k)
historial_costo_base = []

# --- 3. Ciclo de Entrenamiento (SGD) ---
for epoch in range(epochs):

    # Mezclamos los datos en cada época
    indices = np.arange(N)
    np.random.shuffle(indices)

    for i in indices:
        zi = Z_train[i]
        yi = y_train[i]

        # Predicción y cálculo del gradiente
        prediccion = np.dot(w_optimo, zi)
        gradiente_i = (prediccion - yi) * zi + (lambda_base / N) * w_optimo

        # Actualización de pesos (el paso hacia el fondo del tazón)
        w_optimo = w_optimo - learning_rate * gradiente_i

    # Cálculo del costo total al final de la época
    error_cuadratico = np.sum((np.dot(Z_train, w_optimo) - y_train)**2)
    costo_total = (1 / (2 * N)) * error_cuadratico + (lambda_base / 2) * np.sum(w_optimo**2)
    historial_costo_base.append(costo_total)

# --- 4. Impresión del resultado final en consola ---
costo_final = historial_costo_base[-1]
print("-" * 50)
print("ENTRENAMIENTO COMPLETADO")
print("-" * 50)
print(f"Épocas ejecutadas: {epochs}")
print(f"Tasa de aprendizaje: {learning_rate}")
print(f"Costo Total J(w) convergido: {costo_final:.6f}")
print("-" * 50)

# --- 5. Visualización de la Convergencia ---
plt.figure(figsize=(8, 5))
# Agregamos el costo final a la etiqueta (label) para que aparezca en la leyenda
plt.plot(range(1, epochs + 1), historial_costo_base, marker='o', color='blue', markersize=4, lab
plt.title(f"Convergencia del SGD (lambda = {lambda_base})", fontsize=14)
plt.xlabel("Numero de Epocas")
plt.ylabel("Costo Total J(w)")
plt.legend(loc='upper right')
```

```
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
--------------------------------------------------
ENTRENAMIENTO COMPLETADO
--------------------------------------------------
Épocas ejecutadas: 50
Tasa de aprendizaje: 0.01
Costo Total J(w) convergido: 0.006418
--------------------------------------------------
```



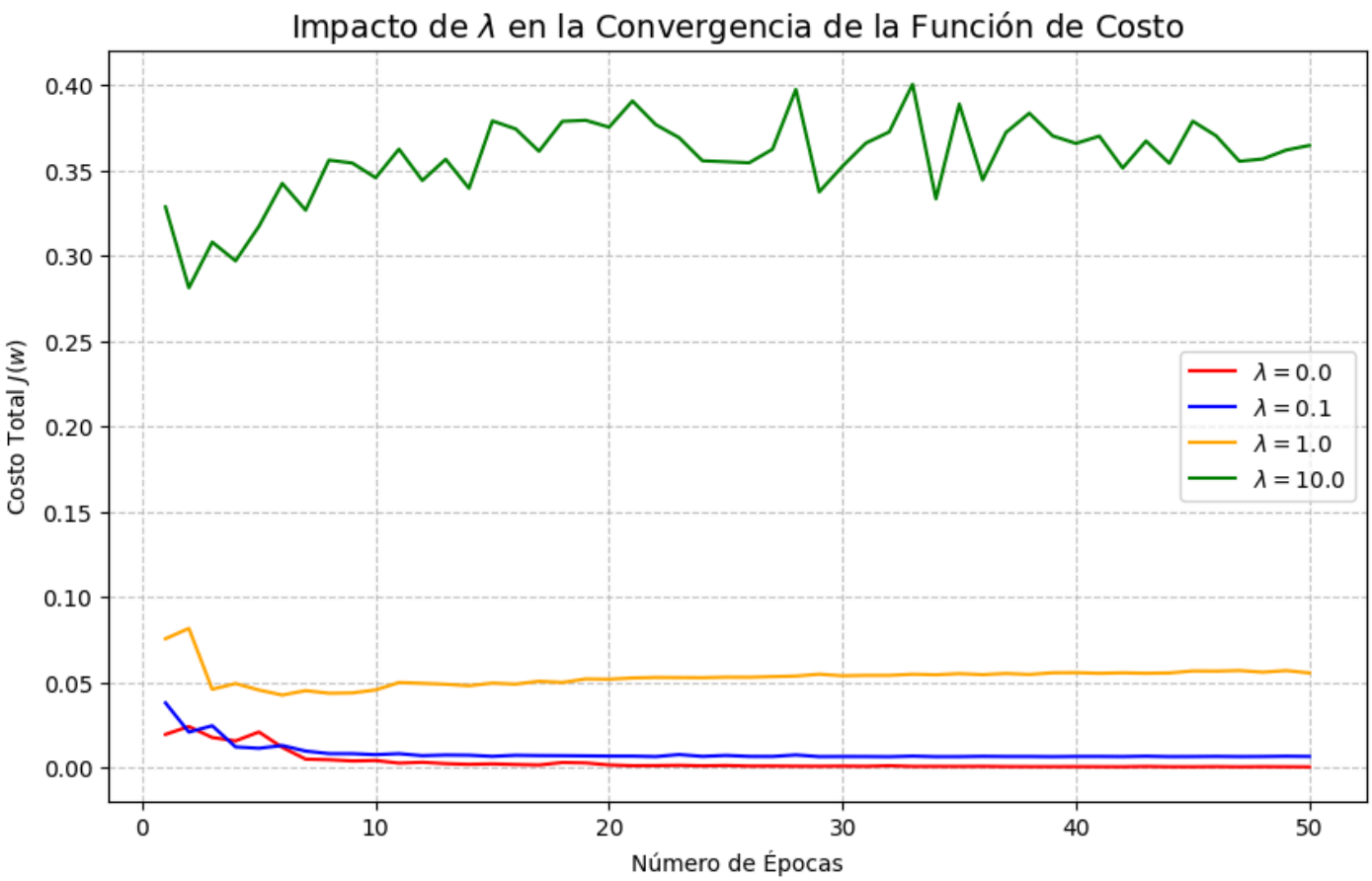
Exploración de la Regularización (Variación de λ)

Para comprender el impacto de la banda elástica de regularización sobre nuestro modelo, repetiremos el proceso de entrenamiento variando el hiperparámetro λ. Evaluaremos escenarios sin regularización ($\lambda = 0.0$), regularización moderada ($\lambda = 0.1, \lambda = 1.0$) y regularización estricta ($\lambda = 10.0$).

Esto nos permitirá observar cómo el término de penalización restringe la magnitud de los pesos y afecta la velocidad y el punto final de convergencia del algoritmo:

In [81]:
```
# --- Exploración de Hiperparámetros ---
valores_lambda = [0.0, 0.1, 1.0, 10.0]
colores = ['red', 'blue', 'orange', 'green']

plt.figure(figsize=(10, 6))

for lam, color in zip(valores_lambda, colores):

    # Reiniciamos los pesos para cada experimento
    w_exp = np.zeros(k)
    historial_exp = []
```

```python
    for epoch in range(epochs):
        indices = np.arange(N)
        np.random.shuffle(indices)

        for i in indices:
            zi = Z_train[i]
            yi = y_train[i]

            # Gradiente y actualización
            prediccion = np.dot(w_exp, zi)
            gradiente_i = (prediccion - yi) * zi + (lam / N) * w_exp
            w_exp = w_exp - learning_rate * gradiente_i

        # Costo total de la época
        error_cuadratico = np.sum((np.dot(Z_train, w_exp) - y_train)**2)
        costo_total = (1 / (2 * N)) * error_cuadratico + (lam / 2) * np.sum(w_exp**2)
        historial_exp.append(costo_total)

    # Graficamos la curva de este lambda
    plt.plot(range(1, epochs + 1), historial_exp, label=f'$\lambda = {lam}$', color=color)

plt.title("Impacto de $\lambda$ en la Convergencia de la Función de Costo", fontsize=14)
plt.xlabel("Número de Épocas")
plt.ylabel("Costo Total $J(w)$")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Tras ejecutar el algoritmo de Gradiente Descendente Estocástico (SGD) a lo largo de 50 épocas, las gráficas generadas nos permiten validar analíticamente el comportamiento de nuestra función de costo bajo

distintos escenarios de regularización Ridge:

- Sin regularización (Línea roja, $\lambda = 0.0$): Presenta el costo final más bajo, acercándose asintóticamente al cero absoluto. Aunque matemáticamente luce ideal, conceptualmente es el escenario de mayor riesgo. Al no existir una penalización sobre la magnitud de los pesos, el modelo goza de total libertad para "memorizar" el ruido de las 16 imágenes de entrenamiento, lo que casi con certeza derivará en un problema de sobreajuste (overfitting).

- Regularización estricta (Líneas naranja y verde, $\lambda = 1.0$ y $\lambda = 10.0$): Aquí evidenciamos el efecto de una "banda elástica" demasiado tensa. Al penalizar tan fuertemente el crecimiento de los pesos, el modelo prioriza mantenerlos cerca de cero antes que corregir sus predicciones. Esto resulta en una convergencia estancada en niveles de error muy altos (costo de ~0.05 y ~0.35 respectivamente). La volatilidad extrema de la línea verde demuestra un modelo que no logra descender al fondo del valle porque la regularización física y matemáticamente no se lo permite, generando subajuste (underfitting).

- $\lambda = 0.1$: Esta curva representa el punto de equilibrio analítico ideal (entre los analizados). Logra descender casi a la par con el modelo sin regularizar, pero conserva un ligero margen de error residual. Ese margen no es una falla, sino el "ajuste" del modelo contra el ruido: ha permitido que los pesos adquieran los valores necesarios para identificar las características clave de los sujetos (Eigenfaces dominantes), pero ha restringido lo suficiente a los demás pesos para evitar la memorización excesiva.

Con base en esta evidencia visual y analítica, descartamos los extremos de subajuste y sobreajuste. Definimos que el vector de pesos w^* obtenido con $\lambda = 0.1$ es el modelo definitivo. Este vector captura la señal real de los datos con el nivel justo de parsimonia.

(c) Validación Fuera de Muestra:

Tras obtener el vector de pesos óptimo (w^*) mediante el entrenamiento con $\lambda = 0.1$, procedemos a evaluar la capacidad de generalización del modelo. Para ello, utilizaremos el conjunto de prueba (`Z_test`), compuesto por 4 imágenes que el algoritmo no ha procesado previamente.

El modelo generará una predicción continua mediante el producto interno entre el vector de pesos y las características de cada nueva imagen ($w^{*T} z_{prueba}$).

Posteriormente, aplicaremos una regla de decisión binaria (función escalón) sobre este resultado continuo:

- Si $w^{*T} z_{prueba} \geq 0$, el modelo clasifica la imagen como la clase 1 (Sujeto 0).
- Si $w^{*T} z_{prueba} < 0$, el modelo clasifica la imagen como la clase -1 (Sujeto 1).

Finalmente, compararemos estas predicciones con las etiquetas reales (`y_test`) para calcular el porcentaje de aciertos (*Accuracy*).

In [83]:
```
import numpy as np

# --- 1. Predicción Continua (El Producto Punto) ---
# Multiplicamos la matriz de prueba (4, 50) por nuestro vector w* (50,)
# Esto nos devuelve un vector de 4 números reales (las "distancias" al hiperplano)
predicciones_continuas = np.dot(Z_test, w_optimo)

# --- 2. Regla de Decisión (Clasificación Binaria) ---
```

```python
# Aplicamos la condición: Si es >= 0 devuelve 1, si no, devuelve -1
predicciones_finales = np.where(predicciones_continuas >= 0, 1, -1)

# --- 3. Cálculo del Porcentaje de Aciertos (Accuracy) ---
# Comparamos nuestras predicciones con las etiquetas reales de prueba
aciertos = (predicciones_finales == y_test)
cantidad_aciertos = np.sum(aciertos)
total_imagenes = len(y_test)

# Calculamos el porcentaje
accuracy = (cantidad_aciertos / total_imagenes) * 100

# --- 4. Reporte de Resultados ---
print("=" * 50)
print("REPORTE DE VALIDACIÓN FUERA DE MUESTRA")
print("=" * 50)
print(f"Predicciones continuas del modelo : {np.round(predicciones_continuas, 3)}")
print(f"Clasificación final del modelo    : {predicciones_finales}")
print(f"Etiquetas reales (Ground Truth)   : {y_test}")
print("-" * 50)
print(f"Imágenes evaluadas : {total_imagenes}")
print(f"Predicciones justas: {cantidad_aciertos}")
print(f"ACCURACY FINAL     : {accuracy:.2f}%")
print("=" * 50)
```

```
==================================================
REPORTE DE VALIDACIÓN FUERA DE MUESTRA
==================================================
Predicciones continuas del modelo : [-0.885 -0.942  0.738  0.936]
Clasificación final del modelo    : [-1 -1  1  1]
Etiquetas reales (Ground Truth)   : [-1 -1  1  1]
--------------------------------------------------
Imágenes evaluadas : 4
Predicciones justas: 4
ACCURACY FINAL     : 100.00%
==================================================
```

Interpretación del resultado (100% Accuracy):
El modelo acertó las 4 imágenes de prueba con alta confianza matemática: las predicciones continuas ($w^{*T} z_{prueba}$) arrojaron valores fuertemente alejados del umbral de decisión 0 (aprox. -0.9 para la clase -1, y $+0.8$ para la clase 1). Esto quiere decir que el modelo no dudó; construyó un margen de separación robusto gracias a la regularización ($\lambda = 0.1$).

Riesgos y Limitaciones:
A pesar del éxito, el modelo presenta tres vulnerabilidades críticas para un entorno real:

- baja significancia estadística: Validar con solo 4 imágenes es insuficiente para garantizar este nivel de generalización a gran escala.
- Sesgo ambiental: Las imágenes de prueba comparten exactamente la misma iluminación y fondo que las de entrenamiento. Ante fotos en entornos no controlados, el rendimiento caerá drásticamente.
- Rigidez espacial: Al depender linealmente de las Eigenfaces, el modelo es altamente sensible a rotaciones de rostro, gestos nuevos u oclusiones parciales (ej. gafas).

(d) Análisis:

Expliquen conceptualmente cómo el término de penalización L2 (el hiperparámetro λ) afecta los valores propios de la matriz Hessiana. ¿Por qué esto matemáticamente garantiza que el problema de optimización sea estrictamente convexo y posea un único mínimo global?

Si entrenamos el modelo sin ninguna regularización ($\lambda = 0$), la curvatura del error depende enteramente de nuestros datos ($H = \frac{1}{N} Z^T Z$). En la práctica, especialmente si hay características redundantes en las imágenes, algunos de los valores propios de esta matriz pueden llegar a ser exactamente cero. ¿Qué significa esto visualmente? Que el fondo de nuestra función de error no es un pozo con un fondo claro, sino un valle completamente plano. En un valle así, existen infinitas combinaciones de pesos con las que el algoritmo se siente igualmente "cómodo", perdiendo la certeza de cuál es el modelo verdaderamente óptimo.

No obstante, al penalizar los pesos con L2, a nivel de derivadas, le estamos sumando al Hessiano la matriz identidad escalada por nuestro hiperparámetro:

$$H_{ridge} = \frac{1}{N} Z^T Z + \lambda I$$

Hacer esa suma nos permite entonces incrementar todos los valores propios originales sumándoles λ.Como nosotros definimos desde el principio un $\lambda > 0$, esos valores que antes podian tomar el valor de cero (y generarnos un valle plano) ahora son si o si positivos (>0 => al tener todos sus valores propios estrictamente mayores a cero la matriz se vuelve definida positiva).

Geométricamente, inyectar λ sería como tomar el valle plano y doblar sus bordes hacia arriba, transformando así el paisaje en un especie de tazón convexo. Como un tazón perfecto tiene un solo punto más bajo, nos garantizamos matemáticamente que nuestro algoritmo no se perderá en mínimos locales ni en soluciones repetidas, sino que aterrizará siempre en un único vector de pesos óptimo (w^*).